

MK_SSL Library - Syntax Review and Development Guidelines

Purpose

This document summarizes all the syntax improvements, Python programming practices, and development patterns needed to make the MK_SSL library professional, maintainable, and ready for pip distribution.

1. General Code Quality Standards

- Follow **PEP8** (Python Enhancement Proposal 8) guidelines throughout the project.
 - Use consistent indentation (4 spaces).
 - No trailing whitespaces.
 - Limit line length to 80-100 characters.
 - Use type hints for all function signatures.
 - Add meaningful docstrings for all classes, methods, and functions.
 - Use `f-strings` for string formatting for better readability and performance.
 - Avoid deep nesting of conditionals, refactor with guard clauses or helper functions.
 - Adopt consistent naming for configuration constants, using uppercase.
-

2. File and Directory Naming

- Use `snake_case` for file names:
 - Example: `multi_model_trainer.py` (instead of `multi-model trainer.py`)
 - Use `PascalCase` for class names.
 - Use `snake_case` for variable, function, and method names.
-

3. Project Structure Recommendation

```
mk_ssl/
  __init__.py
models/
  __init__.py
  clip.py
  slip.py
  albef.py
  simvlm.py
  uniter.py
  vse.py
trainers/
  __init__.py
  multimodal_trainer.py
losses/
  __init__.py
  contrastive_loss.py
  siglip_loss.py
data/
  __init__.py
utils/
  __init__.py
  logger.py
checkpoints/
configs/
setup.cfg
pyproject.toml
README.md
LICENSE
```

4. Import Statements

- Avoid wildcard imports (`from module import *`).
- Always use explicit imports.
- Use relative imports within the package.
- Group imports as:
 1. Standard libraries
 2. Third-party libraries

3. Internal modules

Example:

```
import os
import logging
import torch
import numpy as np
from ak_ssl.models.clip import CLIP
```

5. Logging & Debugging

- Avoid using `print()` statements in production code.
- Use Python's built-in `logging` module for debug/info/warning/error.
- Configure logging with levels and formatting.

Example:

```
import logging
logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.info("Training started")
```

6. Configurations

- Avoid hardcoded values inside functions/classes.
 - Use configuration files (`.yaml` , `.json` , `.toml`) for hyperparameters.
 - Use `argparse` for CLI argument parsing.
 - Support overriding config files via command-line arguments.
-

7. Object-Oriented Design

- Avoid heavy logic inside `__init__()` methods.
- Use separate methods for model building, training, and evaluation.
- Follow **Single Responsibility Principle (SRP)** for each class/module.

- Use inheritance/composition if models share common functionality.
 - Implement abstract base classes for shared interfaces when possible.
 - Use `@staticmethod` or `@classmethod` where appropriate to clarify intent.
 - Use `@property` decorator to safely expose internal attributes that require read-only access.
-

8. Function Design

- Avoid code repetition.
- Prefer reusable helper functions.
- Use meaningful parameter names.
- Avoid deeply nested conditionals.
- Simplify code with guard clauses and ternary operators when safe.
- Use `*args` and `**kwargs` wisely to allow flexible function signatures, but document them properly.
- Always validate arguments early in function execution.
- Prefer keyword-only arguments for clarity in public APIs.
- Avoid mutable default arguments.

Example:

```
def save_model(self, model: nn.Module, save_path: str, *args, **kwargs):  
    """Save the model checkpoint."""  
    torch.save(model.state_dict(), save_path)
```

9. Exception Handling

- Use informative error messages.
- Always specify the context when raising exceptions.
- Catch specific exceptions instead of generic `Exception` whenever possible.
- Use custom exception classes for predictable error flows.
- Add exception chaining where possible (`raise ... from ...`).
- Prefer EAFP (Easier to Ask Forgiveness than Permission) style over LBYL.

Example:

```
try:
    value = obj.attr
except AttributeError:
    value = None
```

10. Constants & Magic Numbers

- Avoid magic numbers in code.
- Use constants at module/class level.
- Document the purpose of constants.
- Centralize constants that are reused across modules.

Example:

```
VALID_INPUT_KEYS = ["input_ids", "attention_mask", "image"]
REGULARIZATION_WEIGHT = 0.1
EPOCH_SAVE_INTERVAL = 5
```

11. Model Checkpointing

- Use `os.path.join()` to handle file paths.
- Ensure versioning and timestamping for saved models.
- Allow for flexible checkpoint loading.
- Validate file existence before attempting to load.
- Provide rollback mechanism if checkpoint loading fails.

12. Training Loop Optimization

- Map methods and optimizers using dictionaries instead of match-case (for Python 3.8+ compatibility).
- Decouple loss functions from training logic and move them into dedicated files under `losses/`.
- Avoid redundant tensor operations inside training loop.
- Add context managers for better resource management (e.g. TensorBoard writer).

- Use learning rate schedulers consistently.
 - Make training loop fully resumable from checkpoints.
-

13. Mixed Precision & Gradient Management

- Continue using `torch.cuda.amp` for mixed precision.
 - Use `clip_grad_norm_` where necessary.
 - Handle scaler updates cleanly and avoid redundant calls.
 - Implement gradient accumulation for large batch sizes.
 - Validate gradients for stability and NaNs.
-

14. Documentation Standards

- Follow Google or NumPy style docstrings consistently.
 - Include Arguments, Returns, Raises, Description, Examples.
 - Maintain a centralized developer documentation for API references.
-

15. Packaging for pip

- Add `pyproject.toml` and `setup.cfg` for pip installation.
 - Include versioning.
 - Follow semantic versioning: `MAJOR.MINOR.PATCH`
 - Build proper `__init__.py` files in each submodule.
 - Test `pip install .` locally before publishing.
 - Write clear `README.md` with usage examples.
-

16. Unit Testing & CI/CD

- Add unit tests for each module.
- Use `pytest` as test framework.
- Setup GitHub Actions or similar CI/CD pipeline.
- Achieve high test coverage (>80%).

- Include tests for edge cases and exception handling.
 - Automate linting and formatting checks in CI.
-

17. Dependency Management

- Use a `requirements.txt` or `poetry` for dependency locking.
 - Avoid version conflicts.
 - Be explicit about CUDA / PyTorch version support.
 - Use dependency groups for optional features.
-

18. Python Version Compatibility

- Target Python 3.8+ for broader compatibility.
- Avoid exclusive use of Python 3.10+ features like `match-case`.
- Use `dataclasses` (introduced in Python 3.7) for configuration classes and model parameters.

Example:

```
from dataclasses import dataclass

@dataclass
class TrainingConfig:
    batch_size: int = 256
    learning_rate: float = 1e-3
```

19. Future Proofing

- Write fully modular code for easy model addition.
 - Allow future additions of: encoders, loss functions, optimizers, schedulers.
 - Use factory patterns to instantiate models dynamically.
 - Build plugin-like architecture for adding models.
-

20. Optional Enhancements

- Add model summaries via `torchsummary`.
 - Add progress bar customization.
 - Allow callbacks/hooks for advanced monitoring.
 - Build experiment management (e.g. `wandb` integration).
 - Add hyperparameter search integrations (Optuna, Ray Tune).
 - Allow multi-GPU training (`DataParallel`, `DistributedDataParallel`).
 - Support model exporting to ONNX, TorchScript.
-

21. Pythonic Programming Practices (Advanced)

- Prefer list/dictionary comprehensions over loops where possible.
- Use context managers (`with` statement) to manage resources.
- Apply unpacking syntax (`*args` , `**kwargs`) responsibly for flexible APIs.
- Prefer immutable data structures (tuples, `namedtuples`, `dataclasses` where `immutable=True`).
- Use generators where appropriate to reduce memory footprint.
- Use `enumerate()` instead of manual index counters:

```
for idx, (batch) in enumerate(data_loader):  
    ...
```

- Leverage built-in functions (`any()` , `all()` , `zip()` , `map()` , `filter()`) for clean code.
- Avoid global variables; use encapsulation.
- Use caching and memoization where computations are redundant (`functools.lru_cache`):

```
from functools import lru_cache  
  
@lru_cache(maxsize=1024)  
def expensive_function(args):  
    ...
```

- Use `@property` decorators to expose read-only computed attributes safely.
- Use `@contextmanager` from `contextlib` for resource-safe custom context managers.
- Prefer EAFP style (`try/except`) over LBYL when accessing attributes or keys:


```
try:
    value = my_dict[key]
except KeyError:
    value = default
```

- Use `functools.partial` to simplify partial function arguments:

```
from functools import partial
new_function = partial(original_function, fixed_argument=10)
```

- Use `__slots__` in small frequently instantiated classes to save memory:

```
class Embedding:
    __slots__ = ('vector', 'label')
    def __init__(self, vector, label):
        self.vector = vector
        self.label = label
```

- Compose small functions that do one thing well, improving testability.
- Use `NamedTuple` or `dataclasses` for clean return types instead of raw tuples:

```
from dataclasses import dataclass

@dataclass
class TrainingResult:
    accuracy: float
    loss: float
```

- Avoid boolean flags in APIs, prefer enums or explicit modes:

```
from enum import Enum

class SaveMode(Enum):
    FAIL_IF_EXISTS = 0
    OVERWRITE = 1

def save_model(path, mode: SaveMode = SaveMode.FAIL_IF_EXISTS):
    ...
```

22. Private Local Variables in Classes

- Private instance variables should be used for:
 - Internal state management (e.g. `_device` , `_optimizer`)
 - Cached intermediate results (e.g. `_cached_embeddings`)
 - Flags and configuration toggles that should not be externally modified (e.g. `_is_initialized`)
 - Sensitive internal values that may change during execution but not be exposed directly (e.g. `_scaler` , `_step_counter`)
 - Any implementation detail that should not form part of the public API.
 - Use single underscore `_variable_name` for conventionally private attributes.
 - Use double underscore `__variable_name` only when name mangling is required (rarely needed).
-